

Unit 01: Introduction to Software engineering

CONTENTS

Objectives

Introduction

1.1 Concepts of Software Engineering

1.2 Types of Software

1.3 Software Characteristics

1.4 Software Myths

1.5 Software Engineering Framework and Process

1.6 software engineering practices

1.7 Software Crisis

Summary

Keywords

Self assessment

Answer for Self Assessment

Review question

Further Reading

Objectives

After studying this unit, you will be able to:

- Discuss various concepts of software engineering.
- Characteristics and types of software
- software process and software engineering practices
- Explain the software engineering challenges and approach

Introduction

User needs and constraints must be determined and explicitly stated before a software product can be developed; the product must be designed to accommodate implementers, users, and maintainers; the source code must be carefully implemented and thoroughly tested; and supporting documents must be prepared. Analysis of change requests, redesign and modification of source code, rigorous testing of the updated code, update of documents to reflect the changes, and dissemination of modified work products to the appropriate user are all examples of software maintenance duties.

In the 1960s, the necessity for methodical approaches to software creation and maintenance became obvious. Cost overruns, timetable slippage, lack of reliability, inefficiency, and lack of user approval plagued many software projects at the time. As computer systems grew larger and more complex, it became clear that demand for software was outpacing our ability to create and maintain it. As a result, the field of software engineering has grown into a significant technological discipline.

In the last four decades, the complexity and nature of software have changed dramatically. Applications of the 1970s used a single processor, took single line inputs, and returned alphanumeric results. Today's programmes, on the other hand, are significantly more complicated, run-on client-server technology, and have a user-friendly interface. They run on a variety of CPUs, operating systems, and even machines from various parts of the world.

The software groups do everything they can to stay on top of fast evolving new technologies while also dealing with development challenges and backlogs. Even the Software Engineering Institute (SEI) recommends that the development process be improved. Change is an unavoidable requirement of the hour. However, it frequently leads to tensions between those who embrace change and others who staunchly adhere to established working methods. As a result, there is a pressing need to adopt software engineering principles, techniques, and strategies in order to avoid conflicts and improve software development in order to provide high-quality software on time and on budget.

Software is a set of instructions to acquire inputs and to manipulate them to produce the desired output in terms of functions and performance as determined by the user of the software. It also includes a set of documents, such as the software manual, meant for users to understand the software system

Engineering on the other hand, is all about developing products, using well-defined, scientific principles and methods

1.1 Concepts of Software Engineering

Evolving Role of Software

In the previous few decades, software's position has shifted dramatically. Hardware, processing architecture, memory, storage capacity, and a wide range of unexpected input and output situations are all improved. As a result of all of these tremendous advancements, increasingly complex and sophisticated computer-based systems have emerged. Although sophistication leads to better results, it can also present issues for those who design these systems.

A team of software experts has taken the place of the lone coder. In order to deliver a complex application, these professionals concentrate on different pieces of technology. However, the specialists are still confronted with the same issues as a lone coder:

Why does it take long to finish software?

Why are the costs of development so high?

Why aren't all the errors discovered before delivering the software to customers?

Why is it difficult to measure the progress of developing software?

All these questions and many more have led to the manifestation of the concern about software and the manner in which it is developed – a concern which lead to the evolution of the software engineering practices.

Software now serves a dual purpose. It is both a product and a vehicle for the delivery of a product. It supplies the computational power embodied by computer hardware or, more broadly, a network of computers accessible by local hardware as a product.

Software is an information transformer, whether it's in a cell phone or a mainframe computer, creating, organizing, obtaining, changing, displaying, or sending data that might be as simple as a single bit or as complicated as a multimedia presentation. Software serves as the vehicle for delivering the product, serving as the foundation for computer control (operating systems), information exchange (networks), and the creation and control of additional programmes (software tools and environments). The most essential product of our day is information, which is delivered by software.

Software transforms personal data (e.g., an individual's financial transactions) to make it more useful in a local context; it manages business information to improve competitiveness; and it serves as a gateway to global information networks (e.g., the Internet) and a means of acquiring information in all of its forms.

The role of computer software has undergone significant change over a time span of little more than 50 years. Dramatic improvements in hardware performance, profound changes in computing architectures, vast increases in memory and storage capacity, and a wide variety of exotic input and output options have all precipitated more sophisticated and complex computer-based systems.

The lone programmer of an earlier era has been replaced by a team of software specialists, each focusing on one part of the technology required to deliver a complex application.

The same questions asked of the lone programmer are being asked when modern computer-based systems are built. Give answers to questions:

1. Why does it take so long to get software finished?
2. Why are development costs so high?
3. Why can't we find all the errors before we give the software to customers?
4. Why do we continue to have difficulty in measuring progress as software is being developed?

1.2 Types of Software

System Software- A collection of programs written to service other programs at system level, For example, compiler, operating systems.

Application software- Developed as per user requirement.

Real-time Software- Programs those monitor/analyze/control real world events as they occur.

Business Software- Programs that access, analyze and process business information

Engineering and Scientific Software-: Software using "number crunching" algorithms for different science and applications, System simulation, computer-aided design.

Embedded Software-: Embedded software resides in read-only memory and is used to control products and systems for the consumer and industrial markets. It has very limited and esoteric functions and control capability.

Artificial Intelligence (AI) Software: Programs make use of AI techniques and methods to solve complex problems. Active areas are expert systems, pattern recognition, games

Internet Software: Programs that support internet accesses and applications. For example, search engine, browser, e-commerce software, authoring tools.

Software Tools and CASE environment: Tools and programs that help the construction of application software and systems. For example, test tools, version control tools.

1.3 Software Characteristics

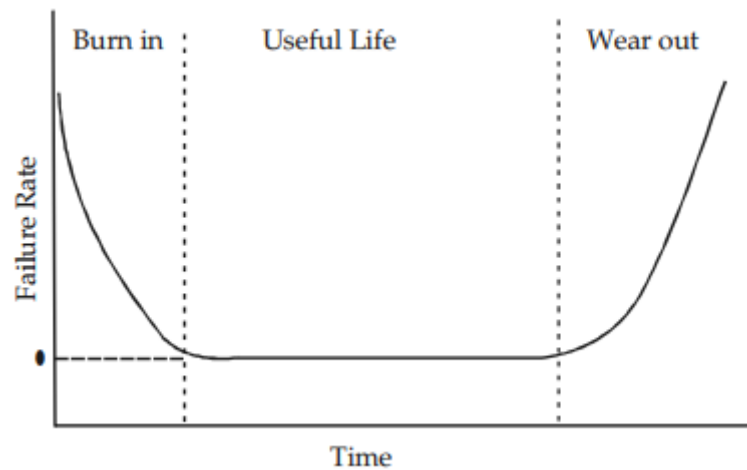
A software product can be judged by what it offers and how well it can be used. In the first NATO conference on software engineering in 1968, Fritz Bauer defined Software engineering as "The establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines". Stephen Schach defined the same as "A discipline whose aim is the production of quality software, software that is delivered on time, within budget and that satisfies its requirements".

Let us now explore the characteristics of software in detail:

Software is Developed or Engineered and not manufactured: Although there exists few similarities between the hardware manufacturing and software development, the two activities differ fundamentally. Both require a good design to attain high quality. But the manufacturing phase of hardware can induce quality related problems that are either nonexistent for software or can be easily rectified. Although both activities depend on people but the relationship between people and work is totally different.

Software does not Wear Out: Figure shows the failure rate of hardware as a function of time. It is often called the "bathtub curve", indicating that a hardware shows high failure at its early stage (due to design and manufacturing defects); defects get resolved and the failure rate reduces to a steady-state level for some time. As time progresses, the failure rate shoots up again as the hardware wears out due to dust, temperature and other environmental factors.

Failure Curve for Hardware



Because the software does not undergo environmental degradation, its failure curve ideally should flatten out after the initial stage of its life. The initial failure rate is high because of undiscovered defects. Once these defects are fixed, the curve flattens at a later stage. Thus, the software does not wear out but deteriorates.

Software is Custom Built and Not Designed Component Wise: Software is designed and built so that it can be reused in different programs. Few decades ago subroutine libraries were created that re-used well defined algorithms but had a limited domain. Today this view has been extended to ensure re-usability of not only algorithms but data structures as well.

Functionality: Refers to the degree of performance of the software against its intended purpose.

Reliability: Refers to the ability of the software to provide desired functionality under the given conditions.

Usability: Refers to the extent to which the software can be used with ease.

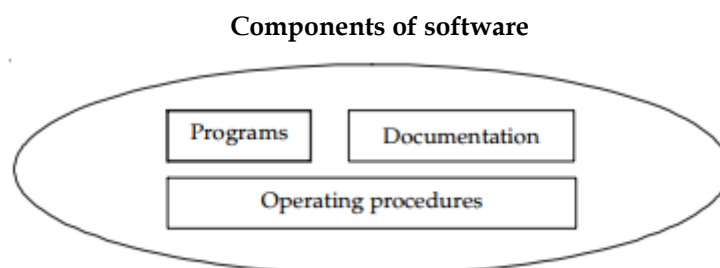
Efficiency: Refers to the ability of the software to use system resources in the most effective and efficient manner.

Maintainability: Refers to the ease with which the modifications can be made in a software system to extend its functionality, improve its performance, or correct errors.

Portability: Refers to the ease with which software developers can transfer software from one platform to another, without (or with minimum) changes. In simple terms, it refers to the ability of software to function properly on different hardware and software platforms without making any changes in it.

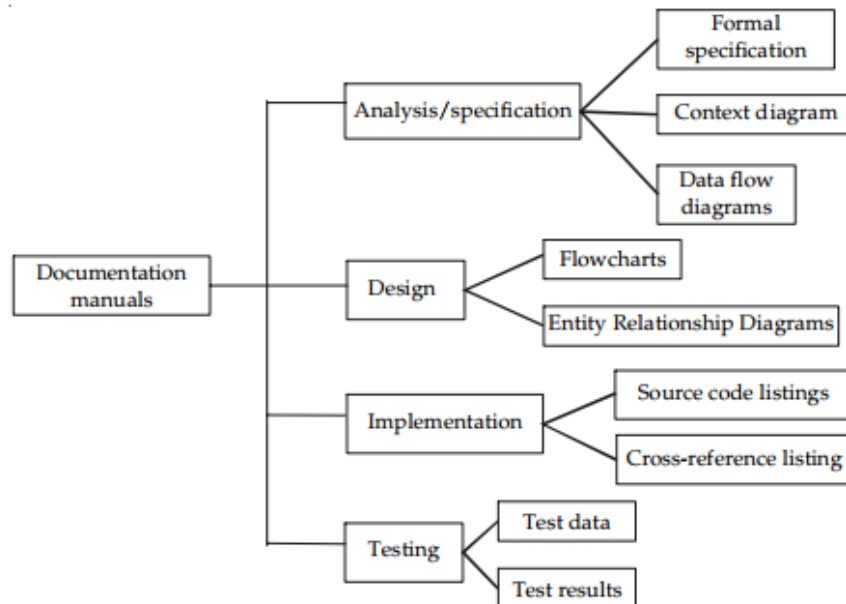
Program vs. Software

Software is more than programs. It comprises of programs, documentation to use these programs and the procedures that operate on the software systems.



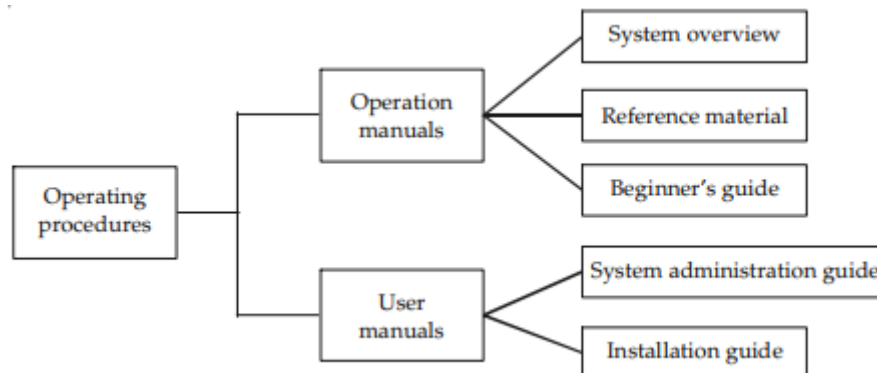
A program is a part of software and can be called software only if documentation and operating procedures are added to it. Program includes both source code and the object code.

List of Documentation Manuals



Operating procedures comprise of instructions required to setup and use the software and instructions on actions to be taken during failure. List of operating procedure manuals/ documents is given in Figure

List of Operating Procedure Manuals



1.4 Software Myths

Myth 1: Computers are more reliable than the devices they have replaced.

Considering the reusability of the software, it can undoubtedly be said that the software does not fail. However, certain areas which have been mechanized have now become prone to software errors as they were prone to human errors while they were manually performed.

Myth 2: Software is easy to change.

Yes, changes are easy to make – but hard to make without introducing errors. With every change the entire system must be re-verified.

Myth 3: If software development gets behind scheduled, adding more programmers will put the development back on track.

Software development is not a mechanistic process like manufacturing. In the words of

Brooks: “adding people to a late software project makes it later”.

Myth 4: Testing software removes all the errors.

SOFTWARE ENGINEERING PRACTICES

Testing ensures presence of errors and not absence. Our objective is to design test cases such that maximum number of errors is reported.

Myth 5: Software with more features is better software.

This is exactly the opposite of the truth. The best programs are those that do one kind of work.

Myth 6: Aim has now shifted to develop working programs.

The aim now is to deliver good quality and efficient programs rather than just delivering working programs. Programs delivered with high quality are maintainable.

The list is unending. These myths together with poor quality, increasing cost and delay in the software delivery have lead to the emergence of software engineering as a discipline.

Types of Myths

Software Myths: Software Myth beliefs about software and the process used to build it – can be traced to the earliest days of computing. Myths have a number of attributes that have made them insidious.

Management Myths: Managers with software responsibility, like managers in most disciplines, are often under pressure to maintain budgets, keep schedules from slipping, and improve quality. Like a drowning person who grasps at a straw, a software manager often grasps at belief in a software myth, if the Belief will lessen the pressure.

Myth: We already have a book that's full of standards and procedures for building software.

Won't that provide my people with everything they need to know?

Reality: The book of standards may very well exist, but is it used?

- Are software practitioners aware of its existence?
- Does it reflect modern software engineering practice?
- Is it complete? Is it adaptable?
- Is it streamlined to improve time to delivery while still maintaining a focus on Quality?

In many cases, the answer to these entire questions is no:

Myth: If we get behind schedule, we can add more programmers and catch up (sometimes called the Mongolian horde concept).

Reality: Software development is not a mechanistic process like manufacturing. In the words of Brooks: "Adding people to a late software project makes it later." At first, this statement may seem counter intuitive. However, as new people are added, people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort.

Myth: If we decide to outsource the software project to a third party, I can just relax and let that firm build it.

Reality: If an organization does not understand how to manage and control software project internally, it will invariably struggle when it outsources software project.

Customer Myths: A customer who requests computer software may be a person at the next desk, a technical group down the hall, the marketing/sales department, or an outside company that has requested software under contract. In many cases, the customer believes myths about software because software managers and practitioners do little to correct misinformation.

Myths led to false expectations and ultimately, dissatisfaction with the developers.

Myth: A general statement of objectives is sufficient to begin writing programs we can fill in details later.

Reality: Although a comprehensive and stable statement of requirements is not always possible, an ambiguous statement of objectives is a recipe for disaster. Unambiguous requirements are developed only through effective and continuous communication between customer and developer.

Myth: Project requirements continually change, but change can be easily accommodated because software is flexible.

Reality: It's true that software requirement changes, but the impact of change varies with the time at which it is introduced. When requirement changes are requested early, cost impact is relatively small.

However, as time passes, cost impact grows rapidly – resources have been committed, a design framework has been established, and change can cause upheaval that requires additional resources and major design modification.

1.5 Software Engineering Framework and Process

It is engineering branch associated with the development of software product using well-defined scientific principles, methods and procedures. The outcome of software engineering is an efficient and reliable software product.

Need of Software Engineering

Large software

Adaptability

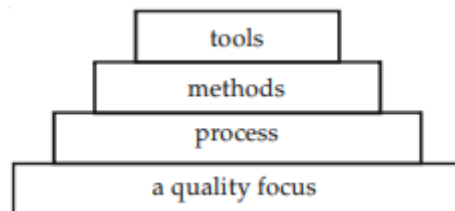
Cost

Dynamic Nature

Quality Management

Software Engineering has a three layered framework. The foundation for software engineering is the process layer. Software engineering process holds the technology layers together and enables rational and timely development of computer software. Process defines a framework for a set of Key Process Areas (KPA) that must be established for effective delivery of software engineering technology.

Software Engineering Framework



Software engineering methods provide the technical knowhow for building software. Methods encompass a broad array of tasks that include requirements analysis, design, program construction, testing, and support. Software engineering tools provide automated or semi-automated support for the process and the methods.

Software Process

A process is a series of steps involving activities, constraints and resources that produce an intended output of some kind.

In simpler words, when you build a product or system, it's important to go through a series of predictable steps – a road map that helps you create a timely, high quality result. The road map that you follow is called a software process.

In the last decade there has been a great deal of resources devoted to the definition, implementation, and improvement of software development processes.

ISO 9000

Software Process Improvement and Capability Determination (SPICE)

SEI Processes

Capability Maturity Model (CMM) for software

Personal Software Process (PSP)

Team Software Process (TSP)

A set of activities whose goal is the development or evolution of software

Generic activities in all software processes are:

Specification: what the system should do and its development constraints

SOFTWARE ENGINEERING PRACTICES

Development: production of the software system

Validation: checking that the software is what the customer wants

Evolution: changing the software in response to changing demands.

Software Process Model

A simplified representation of a software process, presented from a specific perspective.

Example: Process perspectives are:

Workflow perspective – sequence of activities

Data-flow perspective – information flow

Role/action perspective – who does what

Generic Process Models

Waterfall

Evolutionary development

Formal transformation

Integration from reusable components

Software Engineering Methods

Structured approaches to software development which include system models, notations, rules, design advice and process guidance

Rules Constraints applied to system models Notes

Recommendations – Advice on good design practice

Process guidance – What activities to follow

CASE (Computer-Aided Software Engineering)

Software systems which are intended to provide automated support for software process activities.

CASE systems are often used for method support:

Upper- CASE: Tools to support the early process activities of requirements and design.

Lower- CASE: Tools to support later activities such as programming, debugging and testing. Model descriptions – Descriptions of graphical models, which should be produced

Attributes of Good Software

The software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.

Maintainability: Software must evolve to meet changing needs.

Dependability: Software must be trustworthy.

Efficiency: Software should not make wasteful use of system resources.

Usability: Software must be usable by the users for which it was designed.

1.6 software engineering practices

Software engineering practices are a set of approaches to software development.

- To overcome the software development problems.
- Understand the problem (communication and analysis)
- Plan the solution (Modeling and software design)
- Carry out the plan (code generation)
- Examine the result for accuracy (testing and quality assurance)

Understand the problem

- Who has a stake in the solution to the problem?

- What are unknowns?
- Can the problem be compartmentalized?
- Can the problem be represented graphically?

Develop Iteratively

- Critical risks are resolved before making large investments
- Initial iterations enable early user feedback
- Testing and integration are continuous
- Objective milestones provide short term focus

Manage Requirements

- Requirements are dynamic - expect them to change during software development
- User's own understanding of the requirements evolves over time
- Gain agreement with user on what the system should do and not how
- Maintain forward and backward traceability of requirements

Use Component Based Architecture

- Using components permits reuse
- Choice of thousands of commercially available components
- Improved maintainability and extensibility
- Promotes clean division of work among teams of developers

Plan the solution

- Have you seen similar problem before?
- Has a similar problem being solved?
- Can sub problems be defined?
- Can a solution be given for effective implementation?
- Can a design model be created?

Carry out the plan

- Does the solution conform the plan?
- Is the source code traceable to the design model
- Is each component solution correct?
- Have the design and code being reviewed?

Examine the result

- Is the each component part being tested?
- Does the solution produce the result?
- Has the software being evaluated against all stakeholder requirement?

Control Changes to Software

- Without explicit control parallel development degrades to chaos
- Decompose the architecture into subsystems and assign responsibility of each subsystem to a team. Establish secure workspaces for each team i.e. each team is isolated from changes made in other workspaces.
- Establish an enforceable change control mechanism where
- Change requests are prioritized
- Impact of the change request is assessed
- Plan put in place to introduce change in a particular iteration

1.7 Software Crisis

Size: Software is becoming larger and more complex with the growing complexity and expectations out of software. For example, the code in consumer products is doubling every couple of years.

Quality: Many software products have poor quality, i.e., the software produces defects after put into use due to ineffective testing techniques. For example, Software testing typically finds 25 defects per 1000 lines of code.

SOFTWARE ENGINEERING PRACTICES

Cost: Software development is costly i.e., in terms of time taken to develop and the money involved. For example, Development of the FAA's Advance Automation System cost over \$700 per line of code.

Delayed delivery: Serious schedule overruns are common. Very often the software takes longer than the estimated time to develop which in turn leads to cost shooting up. For example, one in four large-scale development projects are never completed.

Summary

- Software is more than programs. It comprises of programs, documentation to use these programs and the procedures that operate on the software systems.
- Any area that involves a sequence of steps (i.e. an algorithm) to generate output can make use of software engineering.
- The software myths together with poor quality, increasing cost and delay in the software delivery have lead to the emergence of software engineering as a discipline.
- Software engineering process holds the technology layers together and enables rational and timely development of computer software.
- The software engineering discipline has been faced with a number of challenges over the years, including those related to quality, management and under estimation.
- One of the challenge for the software engineering discipline is the increasing concern with respect to the design and management of software projects whose complexity is increasing exponentially.
- Legacy systems will also continue to present a considerable challenge to software engineering because the need to ensure that the old technologies are capable to co-exist with the new and vice-versa will be much greater in the near future

Keywords

Artificial Intelligence Software: AI software uses algorithms to solve problems that are not open to simple computing and analysis.

Determinate Program: A program that takes data in a predefined order, applies algorithms to it and generates results is called a determinate program.

Process: A process is a series of steps involving activities, constraints and resources that produce an intended output of some kind.

Program: A program is a part of software and can be called software only if documentation and operating procedures are added to it.

Real-time Software: It refers to the software that controls the events and when they occur.

Software: It is defined as a discipline whose aim is the production of quality software, software that is delivered on time, within budget and that satisfies its requirements.

System Engineering: System engineering is an interdisciplinary field of engineering that focuses on the development and organization of complex artificial systems.

System Software: It is the software program that interacts with the hardware components so that other application software can run on it.

Self assessment

1. Which one is system software?

- A. MS office
- B. Windows XP
- C. Chrome
- D. all of above

2. What are the software crises?

- A. Quality
- B. Size
- C. Cost
- D. All of above

3. Which one is application software _____
- A. Sound drivers
 - B. Operating system
 - C. LAN drivers
 - D. Internet explorer
4. Software is _____
- A. Documentation and configuration of data
 - B. Set of programs
 - C. Set of programs, documentation & configuration of data
 - D. None of the mentioned
5. Characteristics of a software is _____
- A. Operational
 - B. Transitional
 - C. Maintenance
 - D. All of above
6. Software processes resources are _____
- A. SPICE
 - B. CMM
 - C. PSP
 - D. all of above
7. Software process activities include
- A. Quality
 - B. Size
 - C. Cost
 - D. All of above
8. The process of developing a software product using software engineering principles and methods is referred to as _____
- A. Software Engineering
 - B. Software Evolution
 - C. System Models
 - D. Software Models
9. Software engineering practices are _____
- A. Understand the problem
 - B. Plan the solution and Carry out the plan
 - C. Examine the result for accuracy
 - D. all of above
10. SDLC stands for
- A. System Development Life cycle
 - B. Software Design Life Cycle
 - C. Software Development Life Cycle
 - D. System Design Life Cycle

Answer for Self Assessment

- | | | | | |
|------|------|------|------|-------|
| 1. B | 2. D | 3. C | 4. C | 5. A |
| 6. D | 7. D | 8. B | 9. | 10. C |

Review question

1. "Software is designed and built so that it can be reused in different programs." Substantiate with suitable examples.
2. Suppose you are the software engineer of a modern and technically equipped company then explain how software delivers the most important product of our time—information.
3. Critically analyze the role of computer software. "Software has undergone significant change over a time span of little more than 50 years." Comment.
4. "The software differs from hardware as it is more logical in nature and hence, the difference in characteristics." Discuss.
5. Software is easy to change. It is myth? Explain why or why not? Explain with example.
6. Process defines a framework for a set of Key Process Areas (KPA) that must be established for effective delivery of software engineering technology. Analyze this statement.
7. Discuss the concept of software engineering framework.

**Further Reading**

- Books Rajib Mall, Fundamentals of Software Engineering, 2nd Edition, PHI.
- Richard Fairpy, Software Engineering Concepts, Tata McGraw Hill, 1997.
- R.S. Pressman, Software Engineering – A Practitioner's Approach, 5th Edition, Tata McGraw Hill Higher education.
- Sommerville, Software Engineering, 6th Edition, Pearson Education



- ftp://ftp.cordis.europa.eu/pub/ist/docs/directorate_d/st-ds/softeng.pdf
- <http://www0.cs.ucl.ac.uk/staff/A.Finkelstein/talks/10openchall.pdf>
- [http://thepiratebay.sx/torrent/8192581/Software_Engineering__A_Practitioner_s_Approach__7th_Ed._\[PDF\]](http://thepiratebay.sx/torrent/8192581/Software_Engineering__A_Practitioner_s_Approach__7th_Ed._[PDF])
- http://www.etsf.eu/system/files/users/SottileF/XG_Basics_v2.pdf